
My ECE 157B/272B Homework 4

Submitted By Edward Ding

1. The provided code uses a "Replay Memory" to store the agent's experiences and learn from that. **What is the purpose of the Replay Memory?** Why is it important to use a Replay Memory instead of just learning from the agent's most recent experiences?

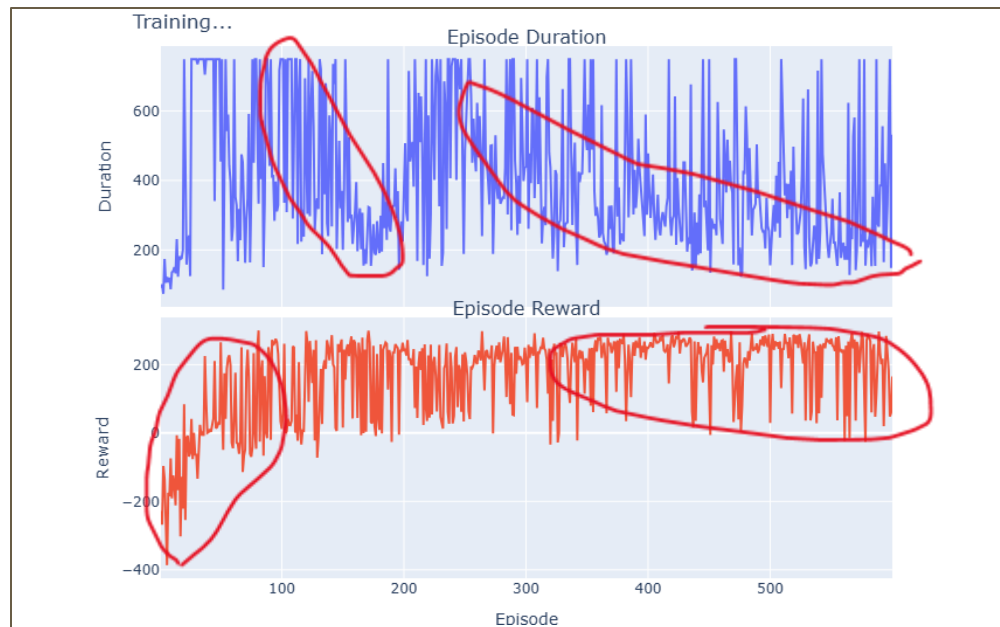
The purpose of the replay memory is to act as a buffer to store past experiences or transitions from the model's interaction with the environment. It allows us to reuse this data later. The main benefit of using something like this is to make sure that the transitions within each batch are decorrelated, which greatly stabilizes and improves the training process. If a model just relearns from most recent experiences, the training data is not independent, which can lead to unstable learning.

2. Submit the hyperparameters used in your final model. Also make a note if you had other training attempts that didn't work out, though you are not required to submit their hyperparameters.

```
# BATCH_SIZE is the number of transitions sampled from the replay
  buffer
# GAMMA is the discount factor as mentioned in the previous section
# EPS_START is the starting value of epsilon
# EPS_END is the final value of epsilon
# EPS_DECAY controls the rate of exponential decay of epsilon, higher
  means a slower decay
# TAU is the update rate of the target network
# LR is the learning rate of the ``AdamW`` optimizer
BATCH_SIZE = 128
GAMMA = 0.99
EPS_START = 0.9
EPS_END = 0.05
EPS_DECAY = 4000
TAU = 0.005
● LR = 1e-4
```

These hyperparameters worked out pretty well, so I didn't change them.

3. Submit the training graphs for your final model. These should show the reward and durations achieved in each episode. If you had other training attempts that didn't work out, you may include their graphs as well, though you are not required to do so. Looking at your final model's training graph, **Mark the eras where the model appears to have differing behaviors.** (E.g. if its average is slowly increasing, then plateaus, or if it's smooth then starts bouncing up and down, sometimes failing.) **About how many eras do you see? How long did it take the model to first really learn to play the game?**



Since our model needs to average around 200 reward points to be considered robust, we see it clearly reach that threshold relatively quickly. Although it might appear at first glance that the rewards plateau, we see that there's still work being done as there's a slight downward trend for the episode duration, meaning that the model gets better at landing the spacecraft in a faster period of time.

4a. Review the videos of your agent playing the game. List behaviors that you see, and when they appear. (Strategies, mistakes...)

At the beginning, the Lunar Lander struggled with even controlling lateral movement; during the first trial, it would move all the way to the left of the screen and fail the test.

The Lunar Lander had trouble figuring out when to turn off its thrusters; sometimes even after landing between the two yellow flags it would still continue burning its thrusters at a lower power, extending the duration of the video.

The Lander would also sometimes tip over from too much lateral thrust, though the model quickly learned not to do this.

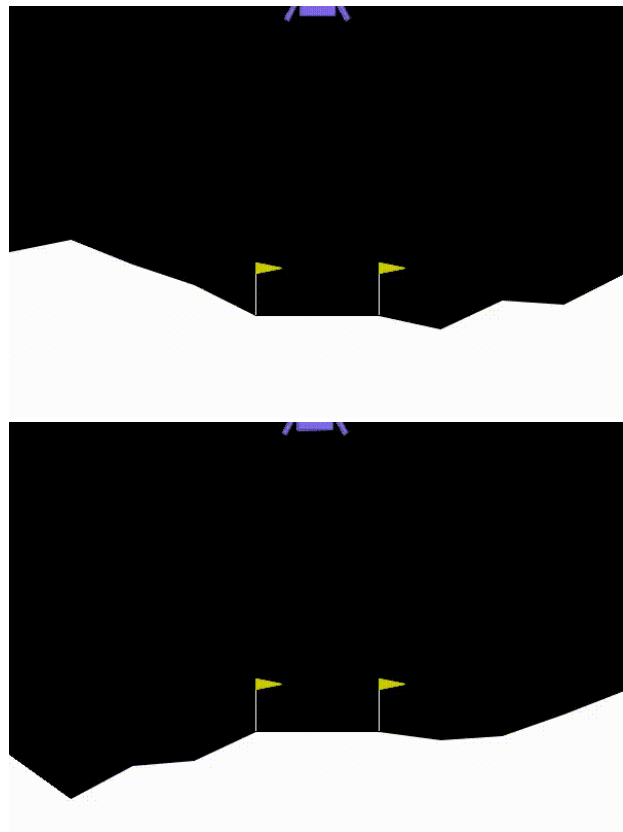
A good run had the Lander descend for a certain distance and then stop, then descend again, self-correcting for mistakes along the way.

4b. Review the videos of your agent playing the game. Does the agent eventually learn to play the game well, or is it too reckless or too cautious?

The model/agent ranged between recklessness and cautiousness when playing the game; however, it generally did worse when it was trying to land too fast as it would cause the lander to go way off target, or drop like a rock. Then again, if it was too conservative it would take way too long for the lander to land, even if it landed in the correct spot. By the end of the 600 epochs the model found a good balance between boldness and caution, maybe leaning towards caution. I would say that it eventually does learn how to play the game decently well.

4c. Review the videos of your agent playing the game. **Do the appearance and disappearance of behaviors in the videos match the eras you marked in the training graph?**

Yes they roughly do; the top GIF is an example of a near-perfect run, where the model was able to land the spacecraft within a short period of time and quite gently as well. This corresponds to the periods in the training graph where there's a plateau when it comes to a low episode duration and a high reward. The bottom GIF is an example of the opposite behavior that's also present, with the model struggling to get the lander between the two flags within a reasonable amount of time.



5. Submit the evaluation duration and reward histograms of your final model, with their means marked. Does the model's policy appear to be consistent, or does it still struggle with the random starting conditions?

The model's policy appears to be very consistent, as there's a clear grouping between 200 and 300 for duration. Additionally, the reward seems to be clustered around the 200-300 scale, with a decently normal distribution as well, which is the targeted accuracy that we need. The outliers in the 700 range and the values below 200 indicate that there's still a little bit of randomness at play; sometimes the model will work really well and sometimes the model will work far less well depending on the starting conditions and if the model gets stuck in a certain part of the gradient.

